SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Industry Problem Solving Task

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	**CO4:** Articulation (BL4, BL5)	**Assessment:**	Assessment Tool 2 – Industry Problem Solving Task
Weightage:	20% of CIA	**Total Marks:**	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	N.MadhuSudhan	**Reg. No.:**	192425303
Section / Batch:	SLOT-C	**Date:**	22/08/2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Explain in detail with sample code if needed.
- Clearly identify all key issues.
- Provide a thorough analysis with validated results and performance evaluation.

Question Type	Focus Area	Questions
Industry Problem Solving Task	Focus on Solving optimization problems using dynamic programming and greedy strategies	Q1

SECTION A – Industry Problem Solving Task

Q1. Ride-Sharing Matching System Optimization (Graph Matching)

A ride-sharing platform must match drivers with passengers in real time to minimize waiting time and travel distance. Analyze how this problem can be modeled using bipartite graph matching. Identify constraints such as driver availability, passenger demand, and location proximity. Design an efficient algorithmic solution to maximize matching efficiency. Discuss scalability issues for large urban environments. Evaluate computational complexity and system responsiveness. Interpret results in terms of service quality and operational efficiency.

Code of Conduct

I N.MadhuSudhan (192425303)_____*(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

*Signature of the Student: N.Madhusudhan*_____

RUBRICS FOR CALCULATION

Industry Problem Solving Task

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%				
Application of Engineering Concepts	25%				
Solution Design	25%				
Use of Tools	15%				
Analysis	10%				
Professionalism	5%				

Total Marks (100):

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature:	Signature:	Signature:
Date:	Date:	Date:

Ride-Sharing Matching System Optimization (Graph Matching)

1. Problem Overview

A ride-sharing platform continuously receives two interleaved streams of events: drivers becoming available at particular locations, and passengers requesting rides from particular pickup points. The platform must repeatedly decide which available driver should serve which waiting passenger so as to minimize passenger waiting time and driver deadhead (empty) travel distance, while keeping the decision latency low enough to feel instantaneous to users. This is naturally modelled as a Bipartite Graph Matching problem, closely related to the classical Assignment Problem studied in Design and Analysis of Algorithms, and solved optimally in polynomial time by the Hungarian (Kuhn–Munkres) algorithm, or approximately in real time by greedy and auction-based heuristics.

2. Graph Modelling

The matching problem is represented as a weighted bipartite graph $G = (D \cup P, E)$:

- D (left vertex set): currently available drivers.
- P (right vertex set): currently waiting passengers / ride requests.
- E (edges): a candidate edge (d, p) exists between driver d and passenger p if d can feasibly serve p (e.g., within a maximum pickup radius or time budget).
- Weight w(d, p): the estimated pickup time (ETA) for driver d to reach passenger p — typically road-network distance × a live traffic multiplier, exactly as travel time is modelled for the shortest-path layer in vehicle-routing systems.

Because every driver can serve at most one passenger per assignment round and every passenger needs exactly one driver, an optimal assignment round corresponds to a minimum-weight bipartite matching (if D and P are equal in size) or a minimum-weight maximum matching (if they are not), which is precisely the Assignment Problem.

3. Constraints

Constraint	Modelling Approach
Driver availability	Only drivers currently marked “free” (not already serving a trip) form the left vertex set D for a given matching round; drivers become unavailable the instant they are matched.
Passenger demand	Each waiting request is a right vertex in P; if requests arrive faster than drivers free up, some requests remain unmatched and are queued for the next round.
Location proximity	An edge (d, p) is only created if the estimated pickup ETA is below a maximum acceptable threshold, keeping the graph sparse and pickups practical.
Vehicle / pool capacity	For ride-pooling variants, a driver already carrying passengers can only be matched to a new passenger if the combined route does not exceed seat capacity or detour-time limits.
Fairness / driver rotation	Some platforms add a soft constraint or edge-weight penalty to avoid always routing requests to the same subset of drivers, distributing income more evenly.

4. Algorithm Design

4.1 ETA Estimation Layer

As in shortest-path-based routing, pairwise driver–passenger travel times are first estimated — in production this uses a precomputed road-network shortest-path index (e.g., Contraction Hierarchies or A* with landmarks) refreshed with live traffic; for the algorithmic core discussed here, Euclidean distance scaled by a traffic multiplier is used as the cost proxy, exactly mirroring how the weighted edges of the matching graph are constructed.

4.2 Matching Layer

Two complementary matching strategies are used together, mirroring the construction-plus-improvement pattern used in vehicle routing:

- Greedy real-time matching: the instant a request arrives, it is assigned to the nearest currently available driver. This gives sub-millisecond response time and is what keeps the app feeling instantaneous, but it is myopic — an early greedy choice can force a later request into a far-away driver.
- Batch-optimal matching (Hungarian / Kuhn–Munkres algorithm): every few seconds, all requests and drivers that are still unmatched are collected into a small batch and matched optimally by solving the assignment problem exactly, minimizing total pickup time across the whole batch in $O(V^3)$ time using the Hungarian algorithm.
- Unweighted alternative — Hopcroft–Karp: when the goal is purely to maximize the number of matched pairs (ignoring distance), the Hopcroft–Karp algorithm finds a maximum cardinality bipartite matching in $O(E\sqrt{V})$, faster than Hungarian when edge weights are not needed.
- Auction algorithm (Bertsekas): a distributed alternative to Hungarian where passengers "bid" for drivers based on ETA; drivers accept the best bid. It converges to a near-optimal assignment and parallelizes naturally across many small city zones.

This two-mode design — instant greedy assignment for responsiveness, periodic optimal batching for efficiency — is the standard pattern used by real-world ride-hailing dispatch systems.

5. Implementation

The following Python program builds a synthetic set of 10 available drivers and 14 waiting passengers, computes the driver–passenger ETA matrix, matches them using the greedy real-time strategy, then re-matches the same batch optimally using the Hungarian algorithm (via scipy's implementation of Kuhn–Munkres), and compares total wait time between the two strategies.

```
import random, math
from scipy.optimize import linear_sum_assignment

random.seed(7)

# ------------------------------------------------------------
# 1. BIPARTITE GRAPH MODEL
# Left set  = available drivers
# Right set = waiting passengers
# Edge weight w(d, p) = estimated travel time (minutes) for driver d to reach passenger p
# ------------------------------------------------------------
def build_city(num_drivers=10, num_passengers=14, area=50):
    drivers = {i: (random.uniform(0, area), random.uniform(0, area)) for i in
range(num_drivers)}
    passengers = {j: (random.uniform(0, area), random.uniform(0, area)) for j in
range(num_passengers)}
    return drivers, passengers

def cost_matrix(drivers, passengers):
    D, P = len(drivers), len(passengers)
    cost = [[0.0] * P for _ in range(D)]
    for i in range(D):
        for j in range(P):
            dx = drivers[i][0] - passengers[j][0]
            dy = drivers[i][1] - passengers[j][1]
            dist = math.hypot(dx, dy)
            traffic_factor = random.uniform(1.0, 1.5)  # congestion multiplier
            cost[i][j] = round(dist * traffic_factor, 2)
    return cost

# ------------------------------------------------------------
```

```
# 2. GREEDY REAL-TIME MATCHING
# Passengers are matched one-by-one, in arrival order, to the
# nearest currently available driver (mirrors an on-demand
# platform matching each request the instant it arrives).
# ------------------------------------------------------------
def greedy_matching(cost, num_drivers, num_passengers):
    available = set(range(num_drivers))
    matches = []
    total_wait = 0.0
    for p in range(num_passengers):
        if not available:
            matches.append((None, p, None))  # queued: no driver free
            continue
        d = min(available, key=lambda d: cost[d][p])
        matches.append((d, p, cost[d][p]))
        total_wait += cost[d][p]
        available.remove(d)
    return matches, total_wait

# ------------------------------------------------------------
# 3. BATCH-OPTIMAL MATCHING (Hungarian / Kuhn-Munkres algorithm)
# Every T seconds, all unmatched drivers and waiting passengers
# accumulated in that window are matched OPTIMALLY (minimum total
# travel time) instead of greedily. This is the standard
# "batching window" technique used by real ride-hailing platforms.
# ------------------------------------------------------------
def optimal_matching(cost, num_drivers, num_passengers):
    row_ind, col_ind = linear_sum_assignment(cost)  # O(n^3) Hungarian algorithm
    matches = []
    total_wait = 0.0
    matched_passengers = set()
    for d, p in zip(row_ind, col_ind):
        matches.append((d, p, cost[d][p]))
        total_wait += cost[d][p]
        matched_passengers.add(p)
    for p in range(num_passengers):
        if p not in matched_passengers:
            matches.append((None, p, None))
    return matches, total_wait

# ------------------------------------------------------------
# 4. RUN SIMULATION
# ------------------------------------------------------------
NUM_DRIVERS = 10
NUM_PASSENGERS = 14
drivers, passengers = build_city(NUM_DRIVERS, NUM_PASSENGERS)
cost = cost_matrix(drivers, passengers)

print("=== Sample Driver -> Passenger Travel-Time Matrix (first driver, first 6 passengers)
===")
print({j: cost[0][j] for j in range(6)}, "...")

print("\n=== Greedy Real-Time Matching ===")
greedy_matches, greedy_total = greedy_matching(cost, NUM_DRIVERS, NUM_PASSENGERS)
matched_g = 0
for d, p, w in greedy_matches:
    if d is None:
        print(f"Passenger {p}: QUEUED (no driver available)")
    else:
        matched_g += 1
        print(f"Driver {d} -> Passenger {p}  ETA={w:.2f} min")
print(f"\nMatched: {matched_g}/{NUM_PASSENGERS}  Total wait time: {greedy_total:.2f} min  "
      f"Avg wait: {greedy_total/matched_g:.2f} min")

print("\n=== Batch-Optimal Matching (Hungarian Algorithm) ===")
opt_matches, opt_total = optimal_matching(cost, NUM_DRIVERS, NUM_PASSENGERS)
matched_o = 0
```

```
for d, p, w in sorted(opt_matches, key=lambda x: x[1]):
    if d is None:
        print(f"Passenger {p}: QUEUED (no driver available)")
    else:
        matched_o += 1
        print(f"Driver {d} -> Passenger {p}  ETA={w:.2f} min")
print(f"\nMatched: {matched_o}/{NUM_PASSENGERS}  Total wait time: {opt_total:.2f} min  "
      f"Avg wait: {opt_total/matched_o:.2f} min")

improvement = (greedy_total - opt_total) / greedy_total * 100
print(f"\n=== Comparison ===")
print(f"Greedy total wait time : {greedy_total:.2f} min")
print(f"Optimal total wait time: {opt_total:.2f} min")
print(f"Improvement (wait/distance reduction): {improvement:.2f}%")
```

6. Output

```
=== Sample Driver -> Passenger Travel-Time Matrix (first driver, first 6 passengers) ===
{0: 45.08, 1: 31.59, 2: 13.6, 3: 35.24, 4: 27.44, 5: 26.55} ...

=== Greedy Real-Time Matching ===
Driver 1 -> Passenger 0  ETA=21.99 min
Driver 9 -> Passenger 1  ETA=20.62 min
Driver 5 -> Passenger 2  ETA=4.74 min
Driver 6 -> Passenger 3  ETA=8.37 min
Driver 3 -> Passenger 4  ETA=8.17 min
Driver 2 -> Passenger 5  ETA=5.46 min
Driver 0 -> Passenger 6  ETA=12.94 min
Driver 7 -> Passenger 7  ETA=4.90 min
Driver 8 -> Passenger 8  ETA=30.45 min
Driver 4 -> Passenger 9  ETA=20.65 min
Passenger 10: QUEUED (no driver available)
Passenger 11: QUEUED (no driver available)
Passenger 12: QUEUED (no driver available)
Passenger 13: QUEUED (no driver available)

Matched: 10/14  Total wait time: 138.29 min  Avg wait: 13.83 min

=== Batch-Optimal Matching (Hungarian Algorithm) ===
Passenger 0: QUEUED (no driver available)
Passenger 1: QUEUED (no driver available)
Driver 5 -> Passenger 2  ETA=4.74 min
Driver 6 -> Passenger 3  ETA=8.37 min
Driver 4 -> Passenger 4  ETA=12.20 min
Driver 2 -> Passenger 5  ETA=5.46 min
Driver 1 -> Passenger 6  ETA=6.41 min
Driver 7 -> Passenger 7  ETA=4.90 min
Driver 9 -> Passenger 8  ETA=6.09 min
Passenger 9: QUEUED (no driver available)
Driver 0 -> Passenger 10  ETA=13.63 min
Passenger 11: QUEUED (no driver available)
Driver 3 -> Passenger 12  ETA=11.04 min
Driver 8 -> Passenger 13  ETA=8.80 min

Matched: 10/14  Total wait time: 81.64 min  Avg wait: 8.16 min

=== Comparison ===
Greedy total wait time : 138.29 min
Optimal total wait time: 81.64 min
Improvement (wait/distance reduction): 40.96%
```

7. Result Interpretation

Metric	Greedy (Real-Time)	Optimal (Hungarian)	Improvement
Matched pairs	10 / 14	10 / 14	0% (driver-limited)
Total wait time	138.29 min	81.64 min	40.96%
Average wait per passenger	13.83 min	8.16 min	40.96%
Unmatched (queued) passengers	4	4	unchanged — bounded by driver supply

With only 10 drivers available for 14 waiting passengers, both strategies necessarily leave 4 passengers queued — no algorithm can match more pairs than the smaller side of the bipartite graph. Where the algorithms differ sharply is efficiency: the batch-optimal Hungarian assignment reduces total passenger wait time by about 41% compared to greedy nearest-driver matching, because greedy commits early requests to the closest driver even when a slightly farther driver would free up a much better match for a later request. On real platforms handling thousands of concurrent requests, even a 10–15% reduction in average wait time is known to measurably improve rider retention and driver utilization; a 41% reduction on this small illustrative batch shows the scale of benefit optimal batching can unlock, particularly during demand surges when greedy myopia is at its worst.

8. Scalability in Large Urban Environments

- Spatial partitioning: the city is divided into a grid or geohash cells so that only drivers within nearby cells are considered candidate edges for a passenger, keeping the bipartite graph sparse instead of fully connected.
- Bounded batch windows: the Hungarian algorithm's $O(V^3)$ cost is only applied to the small batch of requests/drivers accumulated in a short window (e.g., 2–5 seconds) and within one zone, not to the whole city at once.
- Zone-parallel matching: each geographic zone runs its own matching instance independently and in parallel across cores or machines, similar to zone-based CVRP decomposition.
- Hierarchical fallback: if a zone has no available driver, the search radius expands to neighbouring zones before a passenger is left queued.
- Streaming re-optimization: as drivers complete trips and free up, or new requests arrive, the matching graph is incrementally updated rather than rebuilt from scratch.
- At extreme scale (millions of daily rides), approximate methods — auction algorithms, Hopcroft–Karp for cardinality-only matching, or learned dispatch policies — trade a small amount of optimality for the throughput needed to run many matching rounds per second.

9. Computational Efficiency

Component	Algorithm	Time Complexity
ETA / distance computation	Precomputed shortest-path index (Contraction Hierarchies / A*)	Near-constant per query after preprocessing
Greedy real-time matching	Nearest-available-driver heuristic	O(D) per passenger, or O(log D) with a spatial index
Maximum cardinality matching (unweighted)	Hopcroft–Karp	$O(E\sqrt{V})$
Minimum-cost bipartite matching (optimal)	Hungarian / Kuhn–Munkres	$O(V^3)$ per batch
Distributed near-optimal matching	Auction algorithm	$O(V \cdot E / \varepsilon)$ — tunable accuracy/speed trade-off

The overall dispatch pipeline — spatial filtering, greedy instant matching, and periodic small-batch Hungarian optimization — is polynomial and runs comfortably within the sub-second latency budget required for a responsive rider experience, because the expensive $O(V^3)$ step is applied only to small, zone-bounded batches rather than the whole city's drivers and passengers simultaneously. This mirrors the same exact-algorithm-plus-heuristic trade-off used in vehicle-routing systems: exact optimization where the problem size allows it, heuristics where it does not.

10. Conclusion

Modelling ride-sharing dispatch as bipartite graph matching connects an everyday consumer product to well-understood algorithmic foundations: greedy heuristics for instant responsiveness, and the Hungarian algorithm (or Hopcroft–Karp / auction methods at larger scale) for periodically re-optimizing away the inefficiency that greedy assignment leaves on the table. The approach scales to large cities through spatial partitioning, bounded batch windows, and zone-parallel computation, and directly improves the metrics riders and drivers experience — wait time, pickup distance, and driver utilization — without requiring exponential computation, since the exact assignment problem, unlike general vehicle routing, is solvable in polynomial time. Even the modest 12-driver, 14-passenger illustration here shows a ~41% wait-time reduction from optimal batching alone; at true city scale, this kind of algorithmic improvement compounds into materially better service quality and operational efficiency across millions of daily rides.